Machine Learning Step 4

This section describes the machine learning modelling stage of the FURSAN framework. After completing data preprocessing and feature engineering, three modelling approaches were developed: regression models, classification models, and a hybrid model. These approaches were designed to accurately predict satellite collision risk and identify high-risk conjunction events.

4.1 Regression Models

4.1.1 Overview

Regression models were developed to predict continuous collision risk values. This approach enables the FURSAN framework to estimate the probability of satellite collision events using orbital parameters, covariance data, and derived physics-based features. Predicting continuous risk values allows satellite operators to understand the severity of conjunction events and make informed avoidance decisions.

4.1.2 Model Development

Multiple regression algorithms were implemented to evaluate performance and robustness. These models were trained using the engineered features generated during the preprocessing stage. The dataset was split into training and validation sets, and feature scaling was applied before model training.

Models used:

Model	Type	Purpose
Linear Regression	Machine Learning	Baseline regression model for collision probability estimation
Lasso Regression	Machine Learning	Linear model with L1 regularisation for feature selection and preventing overfitting
Random Forest Regressor	Machine Learning	Captures nonlinear relationships using ensemble decision trees
XGBoost Regressor	Machine Learning	Gradient boosting model with strong predictive performance
LightGBM Regressor	Machine Learning	Efficient gradient boosting optimised for large tabular datasets
CatBoost Regressor	Machine Learning	Handles heterogeneous tabular features and categorical interactions
Ridge Regression	Machine Learning	Linear model with L2 regularisation to reduce multicollinearity
Elastic Net	Machine Learning	Combines L1 and L2 regularisation for robust feature selection
Feedforward Neural Network (FNN)	Deep Learning	Basic neural network architecture for probability prediction
Multilayer Perceptron (MLP) Regressor	Deep Learning	Learns nonlinear feature interactions in CDM data
Deep Neural Network (DNN) Regressor	Deep Learning	Deeper architecture capable of modelling complex nonlinear relationships
TabTransformer	Deep Learning	Attention-based architecture designed for tabular datasets
Autoencoder + Regression Head	Deep Learning	Extracts compressed latent features before probability prediction
LSTM Regressor	Deep Learning	Models' temporal evolution of conjunction risk in time-series CDM data
Recurrent Neural Network (RNN)	Deep Learning	Captures sequential dependencies in time-based collision data
GRU (Gated Recurrent Unit)	Deep Learning	Efficient recurrent network for modelling sequential CDM patterns
TabNet	Deep Learning	Deep learning model designed specifically for tabular datasets

Table 1: Regression Models used for Log-Scaled Collision Probability Prediction

4.1.3 Model Evaluation

Several regression models were evaluated and trained to predict the (log-scaled) probability of collision. Model performance is assessed using RMSE, MAE, R^2, and inference time.

Model	RMSE	MAE	R2	Accuracy	Precision	Recall	F1 (Macro)	F2 Score	Runtime (s)
Random Forest	0.114217	0.032259	0.999865	0.998325	0.998366	0.998284	0.998325	0.998301	1845.425973
LightGBM	0.114766	0.053878	0.999864	0.997672	0.997957	0.997386	0.997672	0.9975	1832.949516
XGBoost	0.116792	0.052611	0.999859	0.997672	0.998445	0.996896	0.997672	0.997205	784.234907
CatBoost	0.13537	0.072407	0.999811	0.996937	0.997058	0.996814	0.996937	0.996863	293.465982
MLP Regressor	3.90503	2.419081	0.84272	0.893522	0.917056	0.865289	0.893437	0.875169	2249.081069
Feedforward NN	4.054098	2.650629	0.830483	0.883475	0.908956	0.8523	0.883361	0.863059	186.455197
Deep Neural Network	4.33262	2.456278	0.806391	0.884128	0.89807	0.866596	0.884092	0.872713	374.937518
TabNet	4.426917	2.572978	0.797871	0.87547	0.868683	0.88465	0.875459	0.88141	857.606904
GRU Regressor	6.841178	5.193204	0.517289	0.761599	0.792207	0.709174	0.760941	0.724358	1348.590814
Ridge Regression	6.876644	5.590129	0.512271	0.726148	0.855876	0.543828	0.716727	0.586603	7.305951
Lasso Regression	6.879292	5.59336	0.511896	0.726148	0.855693	0.543992	0.716744	0.586737	91.41622
Elastic Net	6.879406	5.594036	0.511879	0.726229	0.855913	0.543992	0.71682	0.586758	96.119575
RNN Regressor	7.063532	5.342587	0.485401	0.749183	0.765448	0.718487	0.748946	0.727413	257.55454
LSTM Regressor	7.236771	5.761711	0.459849	0.723248	0.791094	0.60665	0.71943	0.636322	775.45305
Linear Regression (Baseline)	9.488637	8.415555	0.071391	0.489585	0.310089	0.017074	0.342868	0.021052	7.468873

Table 2: Regression Models Evaluation Results

Models were compared using:

1. Lowest RMSE
2. Lowest MAE
3. Highest R^2
4. Best F1 / F2
5. Reasonable runtime

Random Forest achieves the best overall regression performance, yielding the lowest RMSE and MAE and the highest R^2 among all evaluated models. Although LightGBM and XGBoost perform similarly, Random Forest provides slightly better error reduction while maintaining very high classification reliability, including the highest accuracy, recall, F1, and F2 scores. Gradient boosting models remain strong alternatives but show marginally higher errors.

Neural network models (MLP, Feedforward NN, and Deep Neural Network) achieve moderate predictive performance but incur significantly higher training and inference costs compared with tree-based models. Sequence-based architectures (RNN, GRU, and LSTM) perform worse despite substantial computational overhead, indicating that temporal memory mechanisms do not substantially improve prediction for this dataset, which primarily consists of tabular orbital features rather than sequential time-series inputs.

Linear models (Linear Regression, Ridge, Lasso, and Elastic Net) show significantly poorer results with much higher error values and low explanatory power, confirming that collision probability prediction exhibits strong nonlinear relationships that cannot be captured effectively by simple linear approaches. Overall, the results demonstrate that ensemble tree-based models, particularly Random Forest, are the most suitable approach for accurate collision probability estimation in this dataset.

4.1.4 Code used:

Code Section 1: Importing Libraries

This section imports all required libraries for regression modelling, evaluation, and deep learning implementation. The libraries include machine learning models such as Linear Regression, Random Forest, XGBoost, LightGBM, and CatBoost, as well as deep learning frameworks such as TensorFlow. Additionally, evaluation metrics such as RMSE, MAE, and R^2 are imported to measure model performance. Parallel processing tools are also imported to speed up model training.

```
import numpy as np
import pandas as pd
import time
import os

from concurrent.futures import ThreadPoolExecutor, as_completed

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    precision_score, recall_score, f1_score, fbeta_score,
    accuracy_score
)

from sklearn.linear_model import (
    LinearRegression, Lasso, Ridge, ElasticNet
)

from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor

from pytorch_tabnet.tab_model import TabNetRegressor

import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (
    Dense, LSTM, GRU, SimpleRNN,
    Input, MultiHeadAttention,
    LayerNormalization, Flatten
)
```

Figure 4.1.4.1.1: Regression Model Imports Code

Code Section 2: Loading Training and Testing Data

This section loads the training and testing datasets used for regression modelling. The training dataset is used to train models, while the testing dataset is used for evaluating performance on unseen data. This ensures fair performance evaluation and prevents overfitting.

```
# Load the training data (used for building the model)
train_data = pd.read_csv('..\\data\\raw\\train_data.csv')

# Load the testing data (used for final evaluation)
test_data = pd.read_csv('..\\data\\raw\\test_data.csv')
```

Figure 4.1.4.2.1: Dataset Loading Code

Code Section 3: CPU Optimisation and Results Storage

This section limits CPU thread usage to prevent system overload during parallel model training. A results list is also created to store model performance metrics. This allows efficient training and organised storage of model evaluation results.

```
# ====================================================
# Prevent CPU oversubscription
# ====================================================
os.environ["OMP_NUM_THREADS"] = "4"

results = []
```

Figure 4.1.4.3.1: CPU Optimisation and Results Storage Code

Code Section 4: Model Evaluation Function

This section defines a custom evaluation function used to measure model performance. The function calculates regression metrics including RMSE, MAE, and R^2. Additionally, classification-based metrics such as accuracy, precision, recall, F1 score, and F2 score are calculated by converting regression predictions into binary values using a threshold. This allows both regression and classification performance evaluation.

```
# ====================================================
# MODEL EVALUATION FUNCTION
# ====================================================
def evaluate_model(name, y_true, y_pred, runtime):

    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    threshold = np.median(y_true)

    y_true_bin = (y_true > threshold).astype(int)
    y_pred_bin = (y_pred > threshold).astype(int)

    accuracy = accuracy_score(y_true_bin, y_pred_bin)
    precision = precision_score(y_true_bin, y_pred_bin, zero_division=0)
    recall = recall_score(y_true_bin, y_pred_bin, zero_division=0)

    f1 = f1_score(y_true_bin, y_pred_bin, average="macro", zero_division=0)
    f2 = fbeta_score(y_true_bin, y_pred_bin, beta=2, zero_division=0)

    results.append({
        "Model": name,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1 (Macro)": f1,
        "F2 Score": f2,
        "Runtime (s)": runtime
    })

    print(f"{name} completed in {runtime:.2f} seconds")
```

Figure 4.1.4.4.1: Model Evaluation Function Code

Code Section 5: Model Runner Function

This section defines a helper function used to train models and generate predictions. The function records training time, fits the model, generates predictions, and sends results to the evaluation function. This ensures consistent evaluation across all models.

```python
# ====================================================
# MODEL RUNNER
# ====================================================
def run_model(name, model, Xtr, Xte, ytr, yte):

    start = time.time()

    model.fit(Xtr, ytr)

    pred = model.predict(Xte)

    runtime = time.time() - start

    evaluate_model(name, yte, pred, runtime)
```

Figure 4.1.4.5.1: Model Runner Function Code

Code Section 6: Target Variable Setup

This section defines the target variable and separates input features from labels. The code automatically detects whether the dataset uses score or risk as the target variable. This improves flexibility and ensures compatibility with different dataset versions.

```python
# ====================================================
# TARGET
# ====================================================
target_train = 'score' if 'score' in train_data.columns else 'risk'
target_test = 'score' if 'score' in test_data.columns else 'risk'

y_train = train_data[target_train]
X_train = train_data.drop(columns=[target_train])

y_test = test_data[target_test]
X_test = test_data.drop(columns=[target_test])
```

Figure 4.1.4.6.1: Target Variable Setup Code

Code Section 7: Categorical Encoding

This section converts categorical features into numerical format using one-hot encoding. The categorical column c_object_type is converted into binary features. The training and testing datasets are aligned to ensure consistency.

```python
# ====================================================
# ENCODING
# ====================================================
categorical_cols = ['c_object_type']

for col in categorical_cols:
    if col in X_train.columns:
        X_train[col] = X_train[col].astype(str)
    if col in X_test.columns:
        X_test[col] = X_test[col].astype(str)

X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)

X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)
```

Figure 4.14.7.1: Categorical Encoding Code

Code Section 8: Data Cleaning

This section cleans the dataset by removing infinite values and replacing missing values using median imputation. Extreme values are clipped to prevent instability during model training. Finally, the dataset is converted into float format.

```
# ====================================================
# CLEAN DATA
# ====================================================
X_train = X_train.replace([np.inf, -np.inf], np.nan)
X_test = X_test.replace([np.inf, -np.inf], np.nan)

median_vals = X_train.median()

X_train = X_train.fillna(median_vals)
X_test = X_test.fillna(median_vals)

X_train = X_train.clip(-1e6, 1e6)
X_test = X_test.clip(-1e6, 1e6)

X_train = X_train.astype(np.float32)
X_test = X_test.astype(np.float32)
```

Figure 4.1.4.8.1: Data Cleaning Code

Code Section 9: Feature Scaling

This section applies feature scaling using StandardScaler. Scaling improves model performance by ensuring features have similar ranges. The scaled dataset is also cleaned to remove NaN values.

```
# ====================================================
# SCALE
# ====================================================
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled = np.nan_to_num(X_train_scaled)
X_test_scaled = np.nan_to_num(X_test_scaled)
```

Figure 4.1.4.9.1: Feature Scaling Code

Code Section 10: Machine Learning Models

This section defines multiple regression models including:

- Linear Regression
- Lasso Regression
- Ridge Regression
- Elastic Net
- Random Forest
- XGBoost
- LightGBM
- CatBoost
- MLP Regressor

These models are trained in parallel using ThreadPoolExecutor to reduce training time.

```python
# ====================================================
# MACHINE LEARNING MODELS
# ====================================================

models = [

    ("Linear Regression (Baseline)", LinearRegression(), X_train, X_test),

    ("Lasso Regression", Lasso(alpha=0.001), X_train_scaled, X_test_scaled),

    ("Ridge Regression", Ridge(alpha=1.0), X_train_scaled, X_test_scaled),

    ("Elastic Net", ElasticNet(alpha=0.001, l1_ratio=0.5), X_train_scaled, X_test_scaled),

    ("Random Forest", RandomForestRegressor(
        n_estimators=300,
        random_state=42,
        n_jobs=-1), X_train, X_test),

    ("XGBoost", XGBRegressor(
        n_estimators=500,
        learning_rate=0.05,
        max_depth=6), X_train, X_test),

    ("LightGBM", LGBMRegressor(
        n_estimators=500,
        learning_rate=0.05), X_train, X_test),

    ("CatBoost", CatBoostRegressor(
        iterations=500,
        learning_rate=0.05,
        depth=6,
        verbose=0), X_train, X_test),
```

```python
    ("MLP Regressor", MLPRegressor(
        hidden_layer_sizes=(128,64),
        max_iter=500,
        random_state=42), X_train_scaled, X_test_scaled)

]

print("\nRunning Machine Learning Models...\n")

with ThreadPoolExecutor(max_workers=8) as executor:

    futures = []

    for name, model, Xtr, Xte in models:
        futures.append(executor.submit(run_model, name, model, Xtr, Xte, y_train, y_test))

    for future in as_completed(futures):
        future.result()
```

Figure 4.1.4.10.1: Machine Learning Models Code

Code Section 11: TabNet Model

This section implements the TabNet deep learning model designed for tabular data. TabNet uses attention-based learning to improve feature representation and model accuracy.

```python
# ====================================================
# TABNET
# ====================================================

print("\nRunning TabNet...\n")
start = time.time()

tabnet = TabNetRegressor()

tabnet.fit(
    X_train.values,
    y_train.values.reshape(-1,1),
    max_epochs=50
)

pred = tabnet.predict(X_test.values).flatten()

evaluate_model("TabNet", y_test, pred, time.time() - start)
```

Figure 4.1.4.11.1: TabNet Model Code

Code Section 12: Deep Learning Models

This section implements multiple deep learning regression models including:

- Feedforward Neural Network
- Deep Neural Network
- LSTM
- RNN
- GRU

These models capture complex non-linear relationships within the dataset and improve prediction accuracy.

```python
# ====================================================
# DEEP LEARNING MODELS
# ====================================================

print("\nRunning Deep Learning Models...\n")

# Feedforward NN
start = time.time()

fnn = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

fnn.compile(optimizer='adam', loss='mse')

fnn.fit(X_train_scaled, y_train, epochs=25, batch_size=32, verbose=0)

pred = fnn.predict(X_test_scaled).flatten()

evaluate_model("Feedforward NN", y_test, pred, time.time() - start)

# Deep Neural Network
start = time.time()

dnn = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])
```

```python
dnn.compile(optimizer='adam', loss='mse')

dnn.fit(X_train_scaled, y_train, epochs=30, batch_size=32, verbose=0)

pred = dnn.predict(X_test_scaled).flatten()

evaluate_model("Deep Neural Network", y_test, pred, time.time() - start)

# LSTM
start = time.time()

X_train_lstm = X_train_scaled.reshape(-1, X_train_scaled.shape[1], 1)
X_test_lstm = X_test_scaled.reshape(-1, X_test_scaled.shape[1], 1)

lstm = Sequential([
    LSTM(32),
    Dense(16, activation='relu'),
    Dense(1)
])

lstm.compile(optimizer='adam', loss='mse')

lstm.fit(X_train_lstm, y_train, epochs=10, batch_size=64, verbose=0)

pred = lstm.predict(X_test_lstm).flatten()

evaluate_model("LSTM Regressor", y_test, pred, time.time() - start)

# RNN
start = time.time()

rnn = Sequential([
    SimpleRNN(32),
    Dense(16, activation='relu'),
    Dense(1)
```

```python
])

rnn.compile(optimizer='adam', loss='mse')

rnn.fit(X_train_lstm, y_train, epochs=10, batch_size=64, verbose=0)

pred = rnn.predict(X_test_lstm).flatten()

evaluate_model("RNN Regressor", y_test, pred, time.time() - start)

# GRU
start = time.time()

gru = Sequential([
    GRU(32),
    Dense(16, activation='relu'),
    Dense(1)
])

gru.compile(optimizer='adam', loss='mse')

gru.fit(X_train_lstm, y_train, epochs=10, batch_size=64, verbose=0)

pred = gru.predict(X_test_lstm).flatten()

evaluate_model("GRU Regressor", y_test, pred, time.time() - start)
```

Figure 4.1.4.12.1: Deep Learning Models Code

Code Section 13: Model Performance Table

This section creates a performance comparison table for all regression models. The results are sorted by RMSE and saved as a CSV file. This table helps identify the best-performing regression model.

```python
# ====================================================
# FINAL TABLE
# ====================================================

regression_df = pd.DataFrame(results)

regression_df = regression_df[[
    "Model",
    "RMSE",
    "MAE",
    "R2",
    "Accuracy",
    "Precision",
    "Recall",
    "F1 (Macro)",
    "F2 Score",
    "Runtime (s)"
]]

regression_df = regression_df.round(6)

regression_df = regression_df.sort_values(by="RMSE")

print("\n============================")
print("MODEL PERFORMANCE COMPARISON")
print("============================")

print(regression_df)

regression_df.to_csv(
    "fursan_model_comparison_regression.csv",
    index=False
)
```

Figure 4.1.4.14.1: Model Performance Table Code